

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA

—o0o—



SINH VIÊN THỰC HIỆN

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: KỸ THUẬT PHẦN MỀM

**HỆ THỐNG QUẢN LÝ CÔNG VIỆC CÁ NHÂN
(PERSONAL WORK MANAGER)**

HÀ NỘI - 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA**

—o0o—

SINH VIÊN THỰC HIỆN

MSSV:

Lớp:

BÁO CÁO BÀI TẬP LỚN

HỌC PHẦN: KỸ THUẬT PHẦN MỀM

**HỆ THỐNG QUẢN LÝ CÔNG VIỆC CÁ NHÂN
(PERSONAL WORK MANAGER)**

GVHD:

HÀ NỘI - 2026

Contents

MỞ ĐẦU	2
1 Giới thiệu	2
1.1 Lý do chọn đề tài và Mô tả bài toán	2
1.2 Mục đích hệ thống	3
1.3 Phạm vi hệ thống	3
1.4 Đối tượng sử dụng	4
1.5 Từ điển thuật ngữ	4
2 Phân tích và thiết kế hệ thống	5
2.1 Phân tích yêu cầu chức năng	5
2.1.1 Quy trình nghiệp vụ	5
2.1.2 Các chức năng của hệ thống	6
2.1.3 Biểu đồ use case	8
2.1.4 Đặc tả các use case	8
2.2 Yêu cầu phi chức năng	11
2.2.1 Yêu cầu về giao diện người dùng (User Interface - UI/UX)	11
2.2.2 Yêu cầu về tính bảo mật (Security & Privacy)	12
2.2.3 Các ràng buộc (Constraints)	12
2.2.4 Các yêu cầu phi chức năng khác (nếu có)	13
2.3 Kiến trúc tổng thể hệ thống	13
2.4 Biểu đồ lớp (Class Diagram)	14
2.5 Thiết kế cơ sở dữ liệu (Local Storage Schema)	15
3 Kết quả thực hiện	16
3.1 Công nghệ sử dụng	16
3.2 Các giao diện chính	18
3.3 Kiểm thử phần mềm	20
3.3.1 Bảng tổng hợp Kịch bản Kiểm thử (Test Cases)	20
3.3.2 Đánh giá kết quả kiểm thử	21
Kết luận	22

Phân công nhiệm vụ

24

Danh mục hình ảnh

MỞ ĐẦU

Trong bối cảnh công nghệ thông tin phát triển mạnh mẽ và áp lực công việc ngày càng gia tăng, việc quản lý thời gian và công việc cá nhân trở thành một nhu cầu thiết yếu đối với mỗi người.

Báo cáo này trình bày toàn bộ quá trình áp dụng các nguyên lý Kỹ thuật phần mềm để phân tích, thiết kế, cài đặt và kiểm thử ứng dụng **Personal Work Manager**. Dự án được kỳ vọng mang lại một công cụ hữu ích, trực quan và hiện đại, giúp người dùng theo dõi tiến độ công việc hàng ngày một cách hiệu quả.

1 Giới thiệu

1.1 Lý do chọn đề tài và Mô tả bài toán

Trong kỷ nguyên công nghệ số 4.0, nhịp sống ngày càng trở nên hối hả, khối lượng công việc và thông tin mà một cá nhân phải tiếp nhận và xử lý mỗi ngày tăng lên theo cấp số nhân. Một người trưởng thành trung bình không chỉ phải đối mặt với các áp lực từ công việc chính, mà còn có các nhiệm vụ học tập, phát triển bản thân, gia đình và các mối quan hệ xã hội. Điều này dẫn đến hiện trạng "quá tải nhận thức", khi mà bộ não con người không thể cùng lúc ghi nhớ và theo dõi sát sao tất cả mọi thứ.

Việc thiếu đi một công cụ quản lý thống nhất và khoa học thường dẫn đến các hậu quả như:

- **Trễ hạn (Missed Deadlines):** Quên mất các công việc quan trọng hoặc không ước lượng đúng thời gian cần thiết để hoàn thành chúng.
- **Mất cân bằng (Work-life Imbalance):** Phân bổ thời gian không đồng đều, dành quá nhiều thời gian cho các tác vụ vô bổ và bỏ bê các mục tiêu dài hạn.
- **Suy giảm hiệu suất:** Bị phân tâm liên tục (Context switching), mất tập trung khi làm việc.

Xuất phát từ thực tiễn đó, bài toán đặt ra là làm thế nào để xây dựng một giải pháp phần mềm hỗ trợ cá nhân ghi chép, lập lịch, nhắc nhở và tối ưu hóa thời gian một cách tự động và trực quan nhất.

1.2 Mục đích hệ thống

Hệ thống **Personal Work Manager** được ra đời với các mục tiêu cốt lõi sau:

- **Số hóa và tập trung hóa thông tin:** Cung cấp một nền tảng duy nhất để người dùng lưu trữ toàn bộ các công việc cần làm (To-do lists), thay thế cho giấy nhớ truyền thống hay các ứng dụng ghi chú rời rạc.
- **Phân loại và phân thứ bậc ưu tiên:** Ứng dụng ma trận Eisenhower để chia công việc thành các mức độ ưu tiên (Cao, Trung bình, Thấp), giúp người dùng luôn biết nên tập trung vào việc gì trước.
- **Tự động hóa luồng công việc:** Tự động sinh ra các công việc lặp lại (hàng ngày, hàng tuần, hàng tháng) mà không cần người dùng phải nhập lại từ đầu, đồng thời tự động phát hiện và cảnh báo các công việc đã quá hạn.
- **Nâng cao năng suất bằng khoa học:** Tích hợp đồng hồ đếm ngược Pomodoro (chia thời gian làm việc thành các block 25 phút) nhằm tối đa hóa sự tập trung và giảm thiểu sự mệt mỏi của não bộ.

1.3 Phạm vi hệ thống

Để đảm bảo tính khả thi và tập trung vào trải nghiệm cốt lõi, phạm vi của hệ thống trong giai đoạn phát triển này được xác định như sau:

- **Nền tảng hoạt động:** Ứng dụng Web tĩnh (Single Page Application - SPA) có khả năng tương thích với mọi trình duyệt web hiện đại trên cả máy tính (Desktop) và thiết bị di động (Mobile).
- **Kiến trúc dữ liệu:** Hệ thống hoạt động độc lập (offline-first) bằng cách tận dụng công nghệ LocalStorage của HTML5. Không sử dụng

Server hay Database tập trung nhằm đảm bảo tính bảo mật và quyền riêng tư tuyệt đối cho dữ liệu cá nhân của người dùng.

- **Phạm vi người dùng:** Hệ thống được thiết kế chuyên biệt cho cá nhân (Single-user mode). Không bao gồm các tính năng phân quyền phức tạp hay làm việc nhóm (Collaborative workspace).

1.4 Đối tượng sử dụng

Hệ thống hướng tới tập khách hàng vô cùng đa dạng, cụ thể bao gồm:

- **Học sinh, sinh viên:** Cần quản lý bài tập về nhà, lịch học thêm, lịch nộp tiểu luận, ôn thi, và các hoạt động ngoại khóa.
- **Nhân viên văn phòng:** Cần ghi chú các cuộc họp, deadline báo cáo, xử lý email, và theo dõi các chỉ số KPI cá nhân.
- **Lập trình viên và Freelancers:** Quản lý các module code cần viết, lịch hẹn với khách hàng, các task fix bug theo độ ưu tiên.
- **Người nội trợ và cá nhân nói chung:** Lập danh sách đi chợ, các công việc dọn dẹp định kỳ (nhắc nhở dọn nhà hàng tuần), lịch khám sức khỏe.

1.5 Từ điển thuật ngữ

Để thuận tiện cho việc theo dõi nội dung của báo cáo, một số thuật ngữ chuyên ngành được giải thích như sau:

- **SDLC (Software Development Life Cycle):** Vòng đời phát triển phần mềm, quy trình chuẩn hóa bao gồm các bước từ khảo sát, phân tích, thiết kế, lập trình đến kiểm thử và bảo trì.
- **SPA (Single Page Application):** Ứng dụng web trang đơn, một cấu trúc web trong đó tất cả các thao tác chuyển trang và kết xuất dữ liệu đều diễn ra tại máy khách (Client) thông qua JavaScript mà không cần tải lại trang.

- **React / Context API:** Thư viện JavaScript của Meta dùng để xây dựng giao diện người dùng. Context API là cơ chế tích hợp sẵn giúp quản lý trạng thái (state) xuyên suốt ứng dụng.
- **LocalStorage:** Bộ nhớ cục bộ được cung cấp bởi trình duyệt web, cho phép lưu trữ dữ liệu vĩnh viễn (lên đến 5MB-10MB) dưới dạng key-value, dữ liệu không bị mất khi tắt máy.
- **Pomodoro Technique:** Phương pháp quản lý thời gian do Francesco Cirillo phát triển, sử dụng đồng hồ để chia nhỏ công việc thành các khoảng thời gian (thường là 25 phút), xen kẽ bởi các khoảng nghỉ ngắn.
- **JSON (JavaScript Object Notation):** Định dạng lưu trữ và truyền tải dữ liệu có cấu trúc nhẹ, dễ đọc đối với cả con người và máy móc.
- **CRUD:** Bốn thao tác cơ sở của mọi hệ quản trị dữ liệu: Create (Tạo), Read (Đọc), Update (Sửa), Delete (Xóa).

2 Phân tích và thiết kế hệ thống

2.1 Phân tích yêu cầu chức năng

2.1.1 Quy trình nghiệp vụ

Trong hệ thống Personal Work Manager, quy trình nghiệp vụ được thiết kế tuân theo nguyên lý "Nắm bắt - Xử lý - Đánh giá" (Capture - Process - Review) dựa trên các phương pháp quản trị thời gian nổi tiếng như Getting Things Done (GTD). Cụ thể, quy trình cốt lõi trải qua các giai đoạn sau:

1. **Giai đoạn Thiết lập (Onboarding):** Khi người dùng khởi động ứng dụng lần đầu tiên, hệ thống sẽ yêu cầu cung cấp thông tin định danh (Họ tên và Địa chỉ Email). Mục đích của giai đoạn này là khởi tạo một không gian lưu trữ (Workspace) mang tính cá nhân hóa, giúp người dùng có cảm giác sở hữu đối với ứng dụng thông qua các lời chào tùy chỉnh (Ví dụ: "Chào buổi sáng, Nguyễn Văn A").

2. **Giai đoạn Ghi nhận (Capture):** Bất cứ khi nào phát sinh một nhiệm vụ mới, người dùng sẽ truy cập tính năng "Thêm công việc". Tại đây, nghiệp vụ yêu cầu người dùng phải phân loại công việc một cách rõ ràng ngay từ đầu thông qua các trường dữ liệu: Tiêu đề công việc, Mức độ ưu tiên (Cao, Trung bình, Thấp), và Thời hạn (Ngày, Giờ).
3. **Giai đoạn Sắp xếp và Lên lịch (Organize & Schedule):** Hệ thống tự động thu thập các nhiệm vụ đã ghi nhận, phân tích thời gian và phân bổ chúng vào Lịch biểu (Calendar) tương ứng. Đồng thời, nghiệp vụ "Lặp lại công việc" (Recurring) được tính toán: nếu một công việc được gán cờ "Hàng ngày", thuật toán sẽ chuẩn bị sẵn kế hoạch để nhân bản công việc đó cho chu kỳ tiếp theo.
4. **Giai đoạn Thực thi (Engage):** Người dùng tiến hành xử lý công việc. Trong lúc làm việc, người dùng có thể kích hoạt tính năng đồng hồ đếm ngược Pomodoro (25 phút) để duy trì sự tập trung. Trong trường hợp công việc trễ hạn (so sánh với thời gian thực của hệ thống), nghiệp vụ tự động chuyển đổi trạng thái của task thành "Quá hạn" và cảnh báo bằng màu đỏ.
5. **Giai đoạn Đánh giá (Review):** Sau khi tích chọn "Hoàn thành" công việc, dữ liệu sẽ được đẩy về hệ thống Thống kê (Dashboard). Tại đây, thuật toán tính toán tỷ lệ hoàn thành (KPI) sẽ đưa ra các phân tích biểu đồ và các lời khuyên động (Dynamic Advices) để giúp người dùng cải thiện hiệu suất cho tuần tiếp theo.

2.1.2 Các chức năng của hệ thống

Dựa trên quy trình nghiệp vụ, hệ thống được chia thành 4 phân hệ (Module) chức năng chính:

A. Module Quản lý Tài khoản và Cấu hình:

- **F-A1. Đăng nhập lần đầu:** Cho phép người dùng nhập Tên và Email để thiết lập hệ thống.

- **F-A2. Chuyển đổi giao diện (Theme):** Cho phép chuyển đổi nhanh giữa chế độ Sáng (Light Mode) và Tối (Dark Mode) để bảo vệ mắt.
- **F-A3. Nhập/Xuất Dữ liệu (Backup & Restore):** Tính năng cho phép tải về toàn bộ cơ sở dữ liệu dưới dạng tệp tin `.json` và tải ngược tệp `.json` lên hệ thống để khôi phục công việc.

B. Module Quản lý Công việc (Task CRUD):

- **F-B1. Thêm công việc mới:** Hỗ trợ form nhập liệu validation ràng buộc tiêu đề.
- **F-B2. Chỉnh sửa công việc:** Cập nhật lại các thông tin của công việc đang chờ.
- **F-B3. Xóa công việc:** Loại bỏ hoàn toàn một công việc khỏi bộ nhớ.
- **F-B4. Cập nhật trạng thái:** Cấu hình thao tác một chạm (One-click) để chuyển trạng thái từ "Chờ" sang "Đang thực hiện" hoặc "Hoàn thành".
- **F-B5. Xử lý công việc lặp lại (Recurring Engine):** Tự động sinh task mới khi task cũ (có chu kỳ) được đánh dấu hoàn thành.
- **F-B6. Tìm kiếm và Lọc:** Lọc công việc theo tổ hợp điều kiện (Trạng thái + Mức độ ưu tiên) và tìm kiếm theo từ khóa.

C. Module Lịch biểu và Thời gian (Calendar & Timer):

- **F-C1. Xem lịch hàng tháng:** Hiển thị lưới lịch 35/42 ô vuông, tự động làm nổi bật ngày hiện tại.
- **F-C2. Bản đồ mật độ công việc:** Đánh dấu các chấm màu dưới ngày biểu thị số lượng và mức độ ưu tiên của công việc trong ngày đó.
- **F-C3. Đồng hồ Pomodoro:** Tiện ích đếm ngược 25 phút làm việc, 5 phút nghỉ ngơi, kèm hiệu ứng âm thanh báo hiệu.

D. Module Thống kê (Analytics):

- **F-D1. Tổng hợp thẻ chỉ số (KPI Cards):** Đếm tổng số công việc, số đã hoàn thành, số đang làm và số quá hạn.
- **F-D2. Biểu đồ tiến độ (Charts):** Vẽ biểu đồ cột và biểu đồ tròn hiển thị phân bố công việc.
- **F-D3. Trợ lý gợi ý (Smart Advisor):** Cung cấp các câu khích lệ hoặc cảnh báo dựa trên tỷ lệ hoàn thành công việc.

2.1.3 Biểu đồ use case

Biểu đồ Use Case là bước sơ đồ hóa các chức năng tương tác giữa người dùng (Actor) và Hệ thống (System Boundary).

(Sinh viên tiến hành thiết kế biểu đồ Use Case tổng quát trên draw.io hoặc StarUML và chèn hình ảnh vào vị trí này. Biểu đồ cần thể hiện một Actor "User" nối với các luồng chức năng như Quản lý Task, Quản lý Lịch, Thống kê, đi kèm các quan hệ <<include>> và <<extend>> hợp lý).

2.1.4 Đặc tả các use case

Dưới đây là tài liệu đặc tả chi tiết (Use Case Specifications) cho 3 chức năng cốt lõi nhất của phần mềm nhằm làm rõ luồng sự kiện (Flow of events) ở cấp độ hệ thống.

1. Đặc tả Use Case: Quản lý đăng nhập lần đầu (Onboarding)

- **ID:** UC-01
- **Tên Use Case:** Đăng nhập và thiết lập cá nhân hóa.
- **Tác nhân (Actor):** Người dùng mới (New User).
- **Mục đích:** Thu thập thông tin tên và email để hệ thống có thể nhận diện và chào hỏi người dùng trong các phiên làm việc tiếp theo.
- **Điều kiện tiên quyết:** Trình duyệt chưa từng lưu trữ dữ liệu user trong LocalStorage.
- **Luồng sự kiện chính (Main Flow):**

1. Hệ thống hiển thị trang màn hình Đăng nhập (LoginPage).
2. Người dùng nhập Tên (Username) và Địa chỉ Email vào các ô văn bản tương ứng.
3. Người dùng nhấn nút "Bắt đầu ngay".
4. Hệ thống kiểm tra dữ liệu đầu vào.
5. Hệ thống đóng gói dữ liệu thành đối tượng JSON và lưu vào LocalStorage với khóa `task-manager-user`.
6. Hệ thống chuyển hướng (Redirect) người dùng tới giao diện Trang chủ (Dashboard).

- **Luồng sự kiện rẽ nhánh (Alternate Flow):**

- Tại bước 4, nếu người dùng bỏ trống Tên hoặc Email, hệ thống từ chối lưu dữ liệu và hiển thị thông báo lỗi màu đỏ: "Vui lòng điền đầy đủ tên và email!".
- Nếu Email không chứa ký tự '@', hệ thống báo lỗi định dạng không hợp lệ.

- **Kết quả (Post-condition):** Hệ thống ghi nhận tài khoản, hiển thị tên người dùng trên thanh tiêu đề và không yêu cầu đăng nhập lại trong tương lai.

2. Đặc tả Use Case: Thêm công việc mới

- **ID:** UC-02
- **Tên Use Case:** Thêm mới công việc vào hệ thống (Create Task).
- **Tác nhân (Actor):** Người dùng hệ thống (User).
- **Mục đích:** Cho phép người dùng ghi nhận một công việc mới vào danh sách.
- **Điều kiện tiên quyết:** Người dùng đã vượt qua bước đăng nhập và đang ở màn hình Dashboard, Tasks hoặc Calendar.
- **Luồng sự kiện chính (Main Flow):**

1. Người dùng nhấn nút "Thêm công việc" (biểu tượng dấu cộng).
2. Hệ thống hiển thị Modal cửa sổ nổi chứa Form thêm công việc.
3. Người dùng nhập Tiêu đề, chọn Mức độ ưu tiên (Cao/Trung bình/Thấp), chọn Ngày thực hiện, Giờ thực hiện và tùy chọn Tần suất lặp lại.
4. Người dùng nhấn nút "Lưu công việc".
5. Hệ thống thu thập thông tin, tự động gán một chuỗi định danh duy nhất (UUID/Timestamp) làm ID cho công việc.
6. Hệ thống so sánh ngày thực hiện với ngày hiện tại để quyết định trạng thái khởi tạo là "Chờ" (Pending) hay "Quá hạn" (Overdue).
7. Cập nhật mảng dữ liệu nội bộ và ghi đè xuống LocalStorage.
8. Đóng Modal và hiển thị công việc mới trên giao diện danh sách.

- **Luồng sự kiện rẽ nhánh (Alternate Flow):**

- Nếu người dùng nhấn ra ngoài vùng tối (Overlay) hoặc nhấn nút "Hủy", Modal sẽ đóng và mọi thông tin chưa lưu sẽ bị xóa bỏ.
- Nếu tiêu đề bị bỏ trống, hệ thống highlight ô tiêu đề yêu cầu nhập liệu.

3. Đặc tả Use Case: Xử lý công việc định kỳ (Recurring Workflow)

- **ID:** UC-03
- **Tên Use Case:** Đánh dấu hoàn thành và sinh công việc lặp lại.
- **Tác nhân (Actor):** Người dùng / Hệ thống (Hỗ trợ ngầm).
- **Mục đích:** Tự động hóa việc duy trì các thói quen hàng ngày bằng cách tự tạo công việc mới khi công việc hiện tại đã xong.
- **Điều kiện tiên quyết:** Trong danh sách tồn tại ít nhất 1 công việc có thuộc tính repeat khác "Không lặp".
- **Luồng sự kiện chính (Main Flow):**

1. Người dùng nhấn vào biểu tượng Checkbox hình tròn của công việc lặp lại.
2. Hệ thống cập nhật trạng thái của công việc đó từ "Chờ" / "Đang thực hiện" thành "Hoàn thành". Giao diện gạch ngang chữ công việc.
3. Hệ thống kiểm tra trường `repeat` của công việc. Phát hiện công việc có thuộc tính lặp.
4. Hệ thống kích hoạt thuật toán `getNextOccurrenceDate()` trong `dateUtils.js` để cộng thêm 1 ngày, 7 ngày hoặc 1 tháng tương ứng vào biến `date`.
5. Hệ thống tự động khởi tạo một đối tượng Task hoàn toàn mới với `date` mới tính toán, giữ nguyên tiêu đề và độ ưu tiên.
6. Hệ thống chèn Task mới lên đầu danh sách tổng và lưu vào `LocalStorage`.

2.2 Yêu cầu phi chức năng

2.2.1 Yêu cầu về giao diện người dùng (User Interface - UI/UX)

Giao diện người dùng là một trong những yếu tố được chú trọng nhất trong ứng dụng Personal Work Manager. Yêu cầu giao diện phải đáp ứng các tiêu chuẩn:

- **Tính thẩm mỹ (Aesthetics):** Áp dụng xu hướng thiết kế Glass-morphism (hiệu ứng kính mờ), sử dụng các dải màu Gradient hài hòa (Soft colors) để giảm thiểu sự căng thẳng cho mắt khi phải làm việc liên tục với danh sách dài.
- **Tính nhất quán (Consistency):** Cấu trúc Layout gồm Sidebar (thanh điều hướng trái) và Main Content (nội dung chính bên phải) phải giữ cố định khi chuyển trang. Các nút bấm (Buttons) mang cùng tính chất phải có màu sắc và hành vi tương tự nhau (Ví dụ: Nút hành động chính luôn là màu Xanh dương đậm).
- **Tính phản hồi (Responsiveness):** Hệ thống CSS sử dụng Grid và Flexbox kết hợp Media Queries để đảm bảo giao diện không bị vỡ

bố cục khi thay đổi kích thước màn hình, từ màn hình Desktop siêu rộng (1080p, 4K) cho đến màn hình nhỏ của điện thoại thông minh (Smartphones).

- **Phản hồi vi mô (Micro-interactions):** Mọi tương tác click chuột, hover (di chuột) lên các thẻ thống kê hay dòng công việc đều phải có hiệu ứng **transition** làm phóng to, đổi màu hoặc bóng đổ mượt mà để tăng cảm giác vật lý cho phần mềm.

2.2.2 Yêu cầu về tính bảo mật (Security & Privacy)

Do tính chất là phần mềm quản lý thông tin cá nhân và lịch trình riêng tư, bảo mật dữ liệu là ưu tiên hàng đầu. Tuy nhiên, thay vì sử dụng các cơ chế mã hóa phức tạp trên Server, hệ thống giải quyết bài toán bảo mật thông qua **Kiến trúc cách ly (Isolation Architecture)**:

- Toàn bộ dữ liệu của người dùng không bao giờ rời khỏi thiết bị cá nhân của họ.
- Ứng dụng không thực hiện bất kỳ lệnh gọi API (Fetch/Axios) nào đến máy chủ bên thứ ba để truyền tải dữ liệu công việc.
- Điều này loại bỏ 100% nguy cơ bị tin tặc tấn công máy chủ trung tâm để đánh cắp cơ sở dữ liệu. Dữ liệu chỉ có thể bị đánh cắp nếu kẻ gian truy cập vật lý trực tiếp vào máy tính của người dùng và mở công cụ DevTools của trình duyệt.

2.2.3 Các ràng buộc (Constraints)

- **Công nghệ phía Máy khách (Client Technology Constraints):** Trình duyệt của người dùng bắt buộc phải hỗ trợ chuẩn ES6 JavaScript và HTML5 Web Storage API. Các trình duyệt lỗi thời như Internet Explorer 11 trở xuống sẽ không thể chạy được phần mềm.
- **Ràng buộc dung lượng (Storage Limit):** LocalStorage trên các trình duyệt tiêu chuẩn (Chrome, Firefox, Edge) giới hạn bộ nhớ phân bổ cho mỗi tên miền (Domain) là 5MB. Vì cấu trúc mỗi công việc

(Task) dưới dạng chuỗi JSON rất nhẹ (khoảng 200 - 300 Bytes), người dùng có thể lưu trữ tối đa lên tới 15.000 - 20.000 công việc. Vượt quá ngưỡng này, hệ thống có thể ném ra ngoại lệ `QuotaExceededError`. Người dùng được khuyến cáo sử dụng tính năng Export để lưu trữ dữ liệu cũ ra ổ cứng máy tính trước khi giới hạn này bị phá vỡ.

2.2.4 Các yêu cầu phi chức năng khác (nếu có)

- **Hiệu năng (Performance & Zero-Latency):** Khác với các ứng dụng Web truyền thống phải đợi Server xử lý và truy vấn SQL (tốn từ 100ms - 500ms), mọi thao tác thêm, xóa, sửa trong ứng dụng này phải có phản hồi ngay tức thì (0ms). Trải nghiệm người dùng phải mượt mà tương đương như đang sử dụng một phần mềm Desktop nguyên bản (Native Desktop App).
- **Tính mở rộng (Scalability - Codebase):** Mã nguồn (Source code) phải tuân thủ chuẩn Clean Code, viết rõ ràng, phân tách Component hợp lý để các thế hệ lập trình viên kế cận dễ dàng bổ sung tính năng mới (Ví dụ: Thêm tính năng làm việc nhóm qua Firebase trong tương lai) mà không phải đập bỏ xây lại từ đầu.

2.3 Kiến trúc tổng thể hệ thống

Dự án được xây dựng dựa trên nguyên lý của kiến trúc **Client-side Component-Based Architecture** (Kiến trúc Dựa trên Thành phần phía Máy khách), một xu hướng phát triển phần mềm vô cùng phổ biến do Facebook đề xuất thông qua thư viện React JS.

Để dễ dàng ánh xạ với các kiến thức Kỹ thuật phần mềm đại cương, chúng ta có thể coi kiến trúc này là một dạng **Mô hình MVC (Model - View - Controller)** rút gọn và hoạt động hoàn toàn tại phía Client.

- **Tầng Data / Model (Lớp Dữ liệu và Trạng thái):** Trong kiến trúc này, "Single Source of Truth" (Nguồn dữ liệu chân lý duy nhất) được đặt tại tập tin `TaskContext.jsx`. Đây là tầng cao nhất trong cây React, đóng vai trò như một kho chứa (Store) bao bọc toàn bộ ứng dụng. Lớp này chịu trách nhiệm định nghĩa cấu trúc của Đối

tượng Người dùng (**user**), Mảng công việc (**tasks**) và duy trì cầu nối liên tục (Observer) với bộ nhớ **LocalStorage** của trình duyệt.

- **Tầng View (Lớp Hiển thị - React Components):** Được cấu trúc từ hàng chục thành phần nhỏ (UI Components) độc lập và có thể tái sử dụng (Reusable). Mỗi màn hình như Dashboard, Calendar, Tasks không phải là một trang HTML riêng rẽ mà được "lắp ghép" từ các mảnh ghép giao diện như **Sidebar.jsx**, **Header.jsx**, **StatsOverview.jsx**, **TaskItem.jsx**. Lớp View tuân thủ nguyên lý "Dumb Component" (Component ngốc): Chúng không tự ý thay đổi dữ liệu mà chỉ làm nhiệm vụ "Lắng nghe" dữ liệu từ Context và "Hiển thị" (Render) ra màn hình thành mã HTML/CSS.
- **Tầng Controller / Action (Lớp Điều khiển và Xử lý Logic):** Bao gồm các hàm xử lý hành động (Action handlers) được khai báo bên trong **TaskContext** như **addTask**, **deleteTask**, **toggleTaskStatus**, **loginUser**. Khi người dùng bấm nút trên lớp View, lớp View không trực tiếp thay đổi cơ sở dữ liệu mà gọi "phái bộ" thông qua các hàm Controller này. Các thuật toán phức tạp như tính toán thời gian, điều phối chuyển đổi ngày giờ được đẩy ra các file tiện ích **utils/dateUtils.js** để cách ly logic nghiệp vụ khỏi giao diện.

Lợi ích của kiến trúc này: Kiến trúc này giúp tối đa hóa khả năng bảo trì (Maintainability) và kiểm thử (Testability). Nếu trong tương lai cần thay đổi logic đánh giá thời gian quá hạn, lập trình viên chỉ cần sửa trong thư mục **utils** hoặc **context** mà không cần đụng đến mã nguồn của lớp giao diện người dùng.

2.4 Biểu đồ lớp (Class Diagram)

Mặc dù JavaScript và React sử dụng tư duy Lập trình hàm (Functional Programming) khá nhiều, nhưng để phục vụ quá trình Phân tích thiết kế hệ thống theo chuẩn OOP (Lập trình hướng đối tượng), hệ thống có thể được trừu tượng hóa thông qua Biểu đồ Lớp.

*(Sinh viên tiến hành vẽ biểu đồ Class Diagram trên công cụ UML. Các Lớp cơ bản bao gồm: **Task** (chứa *id*, *title*, *priority*, *status*...), lớp **User***

(username, email), lớp quản lý *TaskManager* (chứa mảng danh sách các Task và các phương thức CRUD), lớp *PomodoroTimer* (quản lý thời gian), lớp *LocalStorageAdapter* (hỗ trợ đọc ghi dữ liệu). Hệ thống các quan hệ Composition và Dependency cần được vạch rõ.)

2.5 Thiết kế cơ sở dữ liệu (Local Storage Schema)

Vì hệ thống không sử dụng Hệ quản trị Cơ sở dữ liệu quan hệ (RDBMS) như MySQL hay SQL Server, mà dùng Cơ sở dữ liệu phi quan hệ (NoSQL) dựa trên Document/Key-Value của trình duyệt, phần "Thiết kế cơ sở dữ liệu" thực chất là quá trình thiết kế cấu trúc mạng dữ liệu JSON (JSON Schema).

Hệ thống lưu trữ 3 khóa (Keys) chính vào bộ nhớ trình duyệt:

1. Bảng lưu thông tin cấu hình (Settings Table)

- Key: task-manager-theme
- Value: Chuỗi văn bản, chứa giá trị "light" hoặc "dark".

2. Bảng lưu thông tin định danh người dùng (Users Table)

- Key: task-manager-user
- Value: Cấu trúc đối tượng JSON chứa thông tin người dùng đăng nhập lần đầu.
- Ví dụ dữ liệu: {"username": "Nguyễn Văn A", "email": "nva@gmail.com"}

3. Bảng lưu danh sách công việc cốt lõi (Tasks Table)

- Key: task-manager-tasks-v2 (Sử dụng hậu tố v2 để quản lý phiên bản dữ liệu).
- Value: Một mảng (Array) chứa hàng loạt các Đối tượng Công việc (Task Objects). Mỗi Object là một bản ghi (Record) với các trường (Fields) tương tự như các cột trong bảng CSDL quan hệ.
- Cấu trúc Schema của một Task Object:

- `id` (String): Khóa chính (Primary Key), tự sinh bằng UUID hoặc Timestamp.
- `title` (String): Tên của công việc.
- `description` (String): Ghi chú chi tiết (Nullable).
- `date` (String - YYYY-MM-DD): Ngày thực hiện mục tiêu.
- `time` (String - HH:MM): Giờ thực hiện mục tiêu.
- `priority` (String): Mức độ ưu tiên, thuộc tập hợp {"Cao", "Trung bình", "Thấp"}.
- `status` (String): Trạng thái tiến độ, thuộc tập {"Chờ", "Đang thực hiện", "Hoàn thành", "Quá hạn"}.
- `repeat` (String): Tần suất lặp, thuộc tập {"Không lặp", "Hàng ngày", "Hàng tuần", "Hàng tháng"}.
- `createdAt` (String - ISO Date): Thời điểm tạo bản ghi, phục vụ mục đích kiểm toán (Audit).

Việc thiết kế cấu trúc JSON phẳng (Flat structure) thay vì lồng ghép nhiều cấp giúp thuật toán `Array.map()` và `Array.filter()` của React thực thi các thao tác tìm kiếm và render giao diện một cách tối ưu với độ phức tạp $O(n)$.

3 Kết quả thực hiện

3.1 Công nghệ sử dụng

Quá trình xây dựng và phát triển phần mềm được thực hiện bằng cách kết hợp các công nghệ và bộ công cụ hiện đại nhất trong lĩnh vực Lập trình Frontend. Mỗi công cụ được lựa chọn đều trải qua quá trình đánh giá kỹ lưỡng về ưu nhược điểm để phù hợp nhất với kiến trúc của dự án.

1. Thư viện cốt lõi: React (Phiên bản 19)

- **Khái niệm:** React là một thư viện mã nguồn mở chuyên dụng để xây dựng giao diện người dùng (UI) do Facebook phát triển. Điểm mạnh lớn nhất của React là sử dụng Virtual DOM (Mô hình đối tượng tài liệu ảo) thay vì thao tác trực tiếp trên Real DOM.

- **Lý do lựa chọn:** Trong dự án Personal Work Manager, người dùng thường xuyên có các thao tác kéo thả, đánh dấu hoàn thành, thêm/xóa công việc. Thay vì phải tải lại toàn bộ trang web mỗi khi có thay đổi, React chỉ cập nhật chính xác những đoạn mã HTML có sự thay đổi. Điều này tạo ra trải nghiệm "Zero-latency" mượt mà. Hơn thế nữa, phiên bản React 19 cung cấp các Hooks mạnh mẽ (như `useState`, `useEffect`, `useContext`), giúp đơn giản hóa quá trình đồng bộ trạng thái giữa các trang.

2. Công cụ đóng gói và biên dịch: Vite (Phiên bản 8)

- **Khái niệm:** Vite (tiếng Pháp nghĩa là "Nhanh") là một công cụ build tool thế hệ mới dành cho dự án Web hiện đại, thay thế cho công nghệ Webpack truyền thống.
- **Lý do lựa chọn:** Vite tận dụng các mô-đun ES gốc (Native ES modules) của trình duyệt để cung cấp tính năng Hot Module Replacement (HMR) với tốc độ chưa tới 50ms. Điều này cho phép lập trình viên vừa viết mã, vừa xem được giao diện thay đổi ngay lập tức trên trình duyệt. Khi đóng gói sản phẩm (Build Production), Vite sử dụng Rollup để tối ưu hóa mã nguồn, tạo ra các tệp `index.html`, CSS, JS với dung lượng siêu nhẹ, đảm bảo tốc độ tải trang chỉ trong chớp mắt.

3. Ngôn ngữ: JavaScript (Tiêu chuẩn ECMAScript 6+)

- Hệ thống sử dụng cú pháp ES6+ bao gồm Arrow functions (Hàm mũi tên), Destructuring assignment (Phân rã đối tượng), Template literals (Chuỗi mẫu) và Spread operators (Toán tử rải). Những cú pháp này giúp mã nguồn (Source code) trở nên vô cùng ngắn gọn, sạch sẽ và dễ đọc, tuân thủ nguyên lý Clean Code.

4. Hệ thống tạo kiểu (Styling): Custom Vanilla CSS

- Thay vì lạm dụng các thư viện UI công kênh như Bootstrap hay Material-UI, dự án tự xây dựng toàn bộ hệ thống phong cách thiết kế từ đầu (từ file `index.css`).

- Các quy tắc biến hệ thống (CSS Variables) được thiết lập cho toàn bộ bảng màu (Primary, Secondary, Success, Danger), giúp hỗ trợ chuyển đổi giao diện Sáng/Tối (Light/Dark Mode) một cách linh hoạt. Kỹ thuật CSS Grid và Flexbox được áp dụng triệt để nhằm đảm bảo tính Responsive của ứng dụng.

5. Biểu tượng đồ họa: Lucide React

- **Lý do lựa chọn:** Lucide là một nhánh phát triển của Feather Icons, cung cấp hàng ngàn biểu tượng chuẩn vector (SVG) dưới dạng các Component của React. Nhờ đó, icon khi phóng to không bao giờ bị vỡ nét, đồng thời cho phép tùy chỉnh màu sắc và độ dày viền dễ dàng bằng CSS.

3.2 Các giao diện chính

Hệ thống đã triển khai thành công 5 màn hình giao diện cốt lõi. Mỗi màn hình mang một triết lý thiết kế (Design Philosophy) riêng nhằm tối ưu hóa trải nghiệm người dùng.

1. Màn hình Khởi động và Đăng nhập (Onboarding/Login Screen)

- **Mục đích:** Thu thập thông tin định danh của người dùng.
- **Bố cục:** Giao diện tập trung (Centered Layout) với một thẻ đăng nhập đặt chính giữa màn hình (dạng Card). Loại bỏ hoàn toàn thanh điều hướng (Sidebar) và Header để người dùng không bị phân tâm. Nút "Bắt đầu ngay" được làm nổi bật để kích thích thao tác.

(Sinh viên chèn ảnh giao diện Đăng nhập vào đây)

2. Màn hình Bảng điều khiển (Dashboard Overview)

- **Mục đích:** Trở thành trung tâm chỉ huy (Command Center) của phần mềm.
- **Bố cục:** Trang được chia làm 3 tầng. Tầng trên cùng là câu chào cá nhân hóa và thứ, ngày, tháng. Tầng thứ 2 là dãy 4 thẻ thống kê (KPI Cards) cung cấp dữ liệu tức thời về khối lượng công việc. Tầng

dưới cùng được chia tỷ lệ 60-40: Bên trái liệt kê chi tiết các công việc cần làm ngay trong ngày hôm nay; Bên phải là một khung Lịch thu nhỏ giúp người dùng nhận diện khái quát bức tranh toàn cảnh của tháng.

(Sinh viên chèn ảnh giao diện Dashboard vào đây)

3. Màn hình Danh sách Công việc (Tasks List Screen)

- **Mục đích:** Quản trị chuyên sâu (Deep-dive) toàn bộ cơ sở dữ liệu công việc.
- **Bố cục:** Điểm nhấn của màn hình này là thanh công cụ Lọc (Filter Bar). Người dùng có thể kết hợp nhiều trường điều kiện cùng lúc (Ví dụ: Tìm các công việc có trạng thái "Đang thực hiện" VÀ mức ưu tiên "Cao"). Màn hình này cung cấp thanh cuộn dọc (Vertical Scroll) không giới hạn, hỗ trợ hiển thị hàng nghìn công việc mà không làm vỡ khung hình.

(Sinh viên chèn ảnh giao diện Tasks Page vào đây)

4. Màn hình Lịch biểu Toàn cảnh (Full Calendar View)

- **Mục đích:** Lên kế hoạch cho các sự kiện ở thì tương lai.
- **Bố cục:** Một ma trận các ô vuông tương ứng với các ngày trong tháng. Nếu một ngày có công việc mang độ ưu tiên cao (màu đỏ) và độ ưu tiên trung bình (màu cam), bên dưới ô số ngày sẽ xuất hiện các dấu chấm tròn màu sắc tương ứng (Dots indicator). Khi click vào bất kỳ ô nào, danh sách bên cạnh sẽ lập tức cập nhật để hiển thị công việc riêng của ngày đó.

(Sinh viên chèn ảnh giao diện Calendar vào đây)

5. Màn hình Thống kê Phân tích (Analytics & Productivity Stats)

- **Mục đích:** Đánh giá, tổng kết và tạo động lực.
- **Bố cục:** Nổi bật với khối "Trợ lý ảo" ở chính giữa trang. Hệ thống thu thập các chỉ số về tỷ lệ hoàn thành (Completion Rate) để tự động

sinh ra các đoạn văn bản khích lệ. Ví dụ: Nếu tỷ lệ hoàn thành lớn hơn 70%, hệ thống sẽ hiển thị "Tuyệt vời! Bạn đang làm rất tốt!". Nếu có quá 2 công việc bị quá hạn, hệ thống sẽ cảnh báo "Chú ý quản lý thời gian!".

(Sinh viên chèn ảnh giao diện Stats vào đây)

3.3 Kiểm thử phần mềm

Kiểm thử (Testing) đóng vai trò sống còn trong việc khẳng định chất lượng phần mềm trước khi phát hành. Dự án đã thực hiện các bài kiểm thử Hộp đen (Black-box Testing) với kỹ thuật Manual Testing.

3.3.1 Bảng tổng hợp Kịch bản Kiểm thử (Test Cases)

Bảng dưới đây trình bày 10 kịch bản kiểm thử (Test Cases) tiêu biểu nhất được trích xuất từ tài liệu Kế hoạch Kiểm thử (Test Plan):

TC	Chức năng	Các bước thực hiện	Kết quả mong đợi	Trạng thái
1	Đăng nhập hợp lệ	Nhập đầy đủ "Nguyễn Văn A" và email có chứa '@'. Bấm Bắt đầu.	Hệ thống lưu dữ liệu, chuyển thẳng tới Dashboard, không báo lỗi.	Pass
2	Đăng nhập sai	Cố tình để trống trường Email. Bấm Bắt đầu.	Ngăn chuyển trang. Xuất hiện câu cảnh báo màu đỏ yêu cầu nhập liệu.	Pass
3	Thêm mới Task	Nhấn '+', nhập Title, chọn mức độ ưu tiên, chọn ngày, Lưu.	Form tự đóng. Dòng dữ liệu mới xuất hiện trên cùng danh sách.	Pass
4	Xóa (Delete) Task	Nhấn nút biểu tượng Thùng rác trên một dòng công việc.	Task biến mất. Tổng số lượng công việc trên thanh KPI bị giảm đi 1.	Pass
5	Đánh dấu Hoàn thành	Nhấn ô Checkbox của một Task đang "Chờ".	Task bị gạch ngang. Biểu đồ hoàn thành tăng lên. Task bị đẩy xuống dưới.	Pass
6	Sinh Task lặp lại	Hoàn thành một Task có cài đặt Lặp "Hàng ngày".	Tự động sinh ra 1 Task mới y hệt, ngày hết hạn bị cộng thêm 1 ngày.	Pass
7	Quá hạn tự động	Thêm Task có ngày hết hạn bằng ngày của tuần trước.	Task vừa sinh ra tự động khoác lớp áo cảnh báo "Quá hạn" (Đỏ).	Pass
8	Lọc theo Ưu tiên	Mở bộ lọc, chọn hiển thị "Mức độ ưu tiên: Cao".	Các Task Thấp và Trung bình bị ẩn đi, chỉ hiển thị Task có màu Đỏ.	Pass
9	Điều hướng KPI	Tại Dashboard, click chuột vào thẻ "Đang thực hiện".	Tự động chuyển qua trang Tasks List và bộ lọc đã set sẵn "Đang thực hiện".	Pass
10	Xuất tệp tin (Export)	Bấm nút "Export JSON" tại màn hình Thiết lập.	Trình duyệt tự tải xuống tệp tin backup.json chứa toàn bộ DB.	Pass

Table 1: Bảng kết quả kiểm thử chức năng Hệ thống

3.3.2 Đánh giá kết quả kiểm thử

Toàn bộ 10/10 Test Case cốt lõi đều trả về kết quả đạt (Pass). Đặc biệt:

- **Tính toàn vẹn Dữ liệu (Data Integrity):** Thông qua Test Case 3, 4, 5, có thể khẳng định thao tác CRUD hoạt động hoàn hảo, dữ liệu giữa bộ nhớ RAM (React State) và bộ nhớ ổ cứng (LocalStorage) luôn đồng bộ theo thời gian thực mà không gặp lỗi lệch pha (Race conditions).
- **Logic Nghiệp vụ (Business Logic):** Thông qua Test Case 6 và 7, thuật toán xử lý ngày tháng (Date Utils) đã chứng minh được tính chính xác cao. Việc tự động chuyển trạng thái Quá hạn hoạt động đúng với ngày giờ thực tế của hệ điều hành, không bị phụ thuộc vào dữ liệu "chết" ban đầu.

Kết luận

Trải qua nhiều tuần nghiên cứu, thiết kế và lập trình không ngừng nghỉ, dự án **Personal Work Manager** đã chính thức được hoàn thiện và đóng gói thành công. Từ những ý tưởng sơ khai trên giấy về một công cụ giúp tối ưu hóa cuộc sống, nhóm đã từng bước hiện thực hóa nó thành một ứng dụng Web hoàn chỉnh, trực quan và sở hữu hiệu năng xuất sắc.

Trong khuôn khổ của môn học Kỹ thuật phần mềm, dự án không chỉ mang lại một sản phẩm có giá trị sử dụng thực tiễn, mà còn là một bài thực hành vô giá giúp sinh viên hiểu rõ tầm quan trọng của việc tuân thủ quy trình Vòng đời phát triển phần mềm (SDLC). Thay vì lao ngay vào viết code một cách mù quáng, việc dành thời gian cho phân tích Use Case, thiết kế kiến trúc Component, xây dựng lược đồ JSON và viết Test Case đã giúp ngăn chặn rủi ro kiến trúc và đảm bảo tiến độ dự án đi đúng hướng.

1. Những điểm sáng của hệ thống (Ưu điểm)

- **Kiến trúc linh hoạt:** Sự kết hợp giữa React 19 và Vite tạo ra một nền tảng vững chắc. Nhờ việc sử dụng Context API (Singleton Pattern) và Custom Hooks, hệ thống tránh được tình trạng mã nguồn bị "spaghetti" (chồng chéo), giúp dễ dàng bảo trì và bổ sung tính năng mới.

- **Hiệu năng tuyệt đối (Zero-latency):** Khác biệt lớn nhất của hệ thống là không sử dụng cơ sở dữ liệu trên mây mà tương tác trực tiếp với LocalStorage của trình duyệt. Mọi thao tác Thêm, Sửa, Xóa đều cho phản hồi ngay lập tức (0 mili-giây), mang lại trải nghiệm tiệm cận với một ứng dụng Native trên máy tính cài đặt sẵn.
- **Thiết kế Thẩm mỹ (UI/UX):** Không lạm dụng các thư viện giao diện công nghiệp khô khan, giao diện được tùy biến thủ công với CSS thuần (Vanilla CSS) áp dụng hiệu ứng Glassmorphism. Màu sắc được tinh chỉnh để giảm mỏi mắt, các khối thông tin được phân cấp rõ ràng theo mức độ ưu tiên.

2. Tồn tại và Hạn chế (Nhược điểm)

- Mặc dù LocalStorage mang lại hiệu năng siêu việt, nhưng nó cũng là con dao hai lưỡi. Nhược điểm lớn nhất là tính "cục bộ" (Local). Người dùng không thể vừa thêm một công việc trên máy tính ở cơ quan, rồi về nhà dùng điện thoại cá nhân để check-off công việc đó (Trừ khi họ tự mình export và import file thủ công qua Email hoặc USB).
- Hệ thống hiện tại chưa hỗ trợ tính năng làm việc nhóm (Collaborative mode) hay ủy quyền công việc (Delegate) cho người khác.

3. Hướng phát triển trong tương lai (Future Work) Để biến Personal Work Manager từ một dự án bài tập lớn thành một sản phẩm thương mại hóa (SaaS - Software as a Service), quy trình SDLC đề xuất các bước tiến hóa (Evolution) sau cho các phiên bản tiếp theo:

- **Xây dựng tầng Backend (API Server):** Sử dụng Node.js (Express/NestJS) kết hợp cùng cơ sở dữ liệu MongoDB hoặc PostgreSQL. Khi đó, LocalStorage sẽ chỉ còn đóng vai trò là bộ đệm (Cache), mọi dữ liệu chính thức sẽ được đồng bộ lên Cloud để hỗ trợ đăng nhập đa thiết bị (Multi-device Sync).
- **Xác thực OAuth 2.0:** Cho phép người dùng đăng nhập nhanh bằng tài khoản Google, Facebook, Apple thay vì chỉ nhập tên thủ công như hiện tại.

- **Tích hợp AI Trợ lý:** Bổ sung thêm công nghệ Trí tuệ nhân tạo (như ChatGPT API) để hệ thống tự động đọc tiêu đề công việc và gợi ý thời gian hoàn thành (Estimation), hoặc tự động chia nhỏ một mục tiêu lớn (Epic) thành các công việc nhỏ (Sub-tasks).

Phân công nhiệm vụ

Sự thành công của dự án là kết quả của quá trình làm việc nhóm hiệu quả, áp dụng phương pháp luận Agile. Bảng dưới đây tóm tắt khối lượng công việc đã được phân bổ và thực thi bởi các thành viên:

STT	Họ và Tên	Chi tiết Nhiệm vụ đảm nhiệm	Tỉ lệ đóng góp	Đánh giá
1	Sinh viên A (Trưởng nhóm)	- Khảo sát yêu cầu, vẽ biểu đồ Use Case/Class. - Setup cấu trúc thư mục React và Vite ban đầu. - Lập trình Module xác thực (On-boarding). - Lập trình Module Thống kê (KPI & Charts). - Tổng hợp viết tài liệu Báo cáo LaTeX.	35%	100%
2	Sinh viên B	- Thiết kế toàn bộ giao diện CSS (UI/UX). - Lập trình Module Quản lý Task (CRUD). - Xây dựng thuật toán Kéo Thả (Drag & Drop). - Tích hợp tính năng Xuất/Nhập dữ liệu JSON.	35%	100%
3	Sinh viên C	- Nghiên cứu và code tiện ích Pomodoro Timer. - Xây dựng thuật toán tính toán Lịch (Calendar). - Phát triển thuật toán sinh công việc định kỳ. - Viết kịch bản kiểm thử (Test Cases) & Test lỗi.	30%	100%

Table 2: Bảng phân công nhiệm vụ và khối lượng công việc của nhóm